



Fundamentals of Programming: Control Structure

Bilal Ahmed

Agenda

- Introduction to control structure
- If statement
- If-else statement
- If-else-If statement
- Nested If
- Switch Case Statement
- Loops
 - For Loop
 - While Loop
 - Do While Loop
 - Foreach Loop (After Arrays)
 - Continue, Break, goto statement

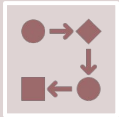
Flow of execution

- ▶ The order in which a program executes
- ▶ The Normal flow of program execution is sequential

Control Statements

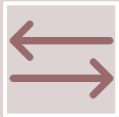
- ▶ Control the flow of execution of a Program
- ▶ Determine how instructions will be executed based on a certain criteria/condition

Types of Control Statements



Sequential:

Default flow of program, without any decision making



Selection/Conditional:

Make decisions and choose different path based on conditions/expressions



Repetition/Loop Statements:

Perform a task repeatedly

Need of Conditions

6

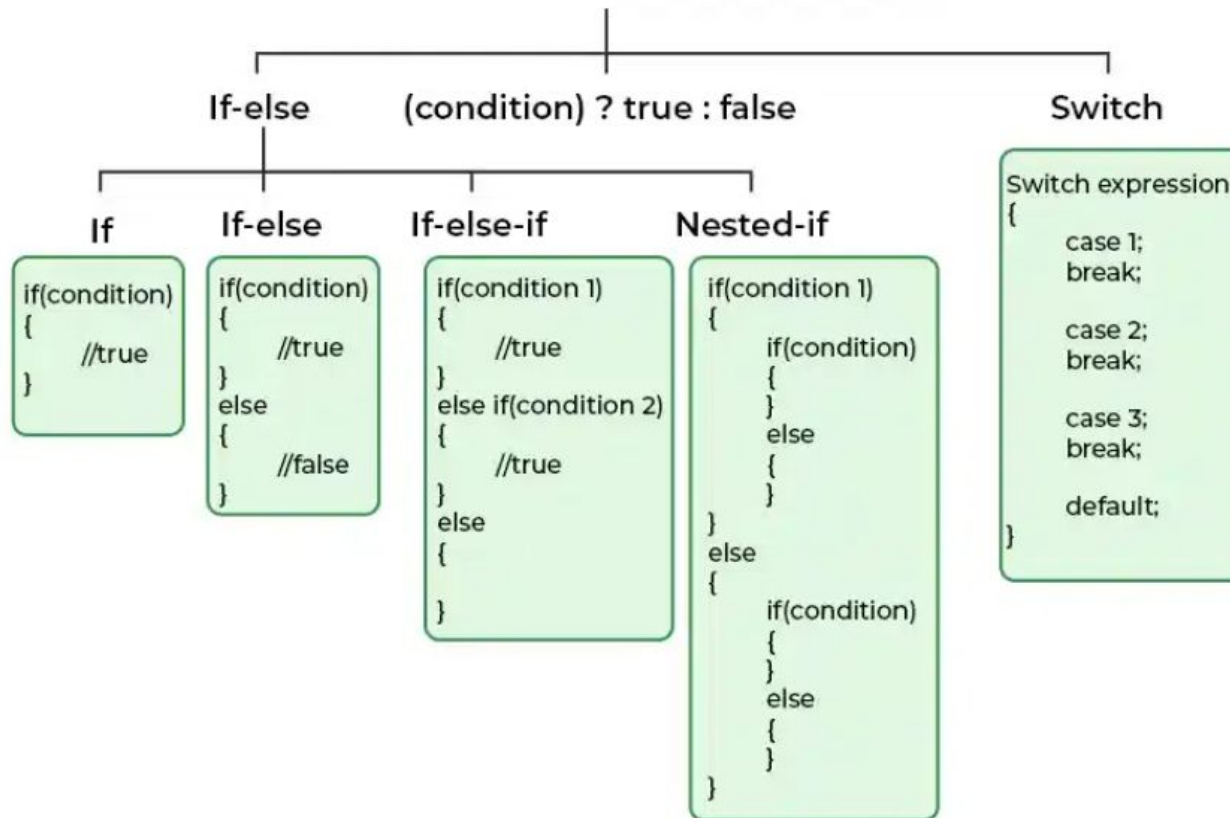


In Real Life we make decisions and take actions accordingly

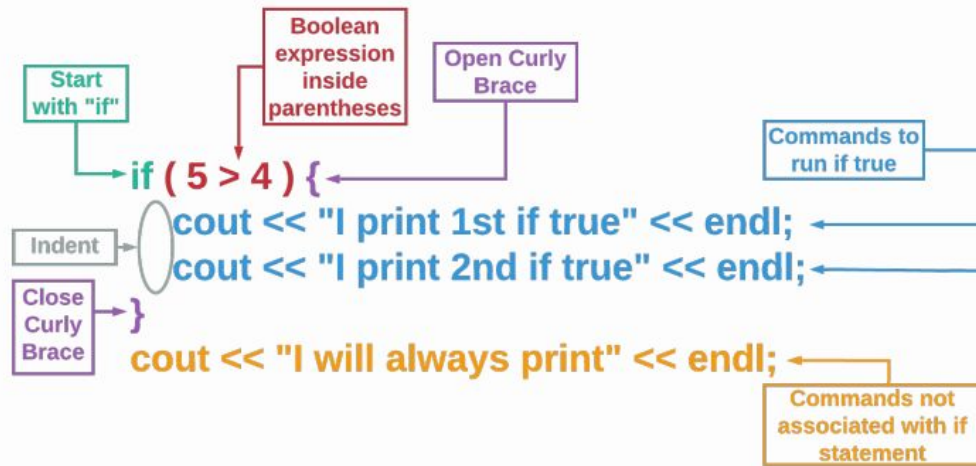


Likewise in programming sometimes we may have to take decisions

Conditional Statements in C



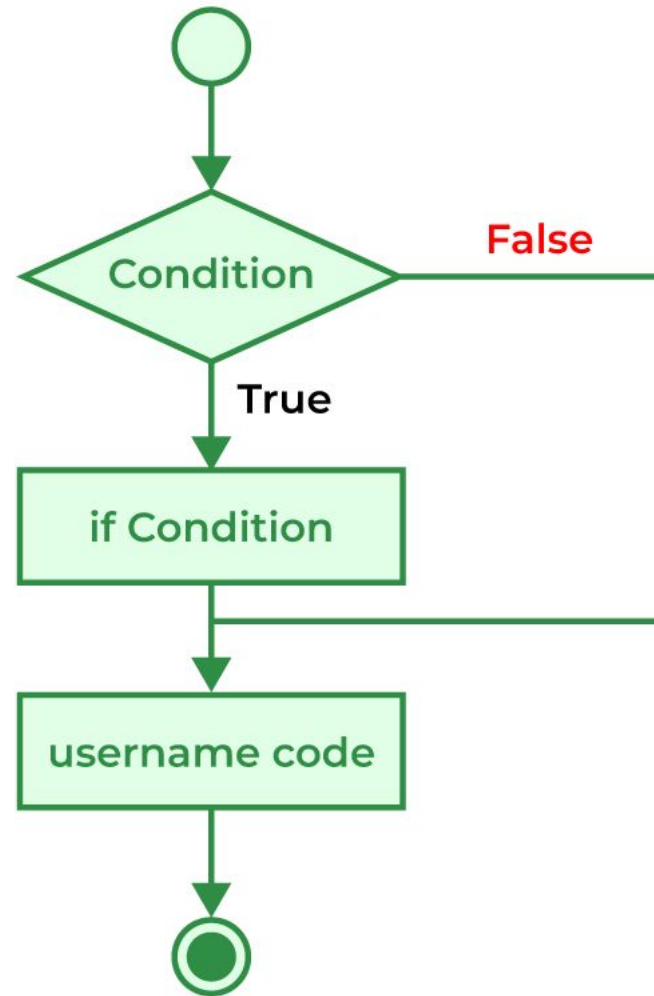
Types of
Selection/Conditional
Statements



If-Statement

If statements in C++ must contain the following items:

- * The keyword `if`.
- * A boolean expression in parentheses, `()`.
- * Curly braces, `{}`, surrounding all lines of code that will run if the boolean expression is true.



Flowchart of If-Statement

```
#include <iostream>
using namespace std;

int main() {

    if (7 > 5) {    // condition
        cout << "Statement if condition is True ";
    }
    return 0;
}
```

Example:

Testing Multiple Cases:

11



Sometimes you will need to test your variable for multiple times



Be sure that you have set up your conditional to test for all possible values

```
#include <iostream>
using namespace std;

int main() {
    int grade = 90;

    if (marks > 70) {
        cout << "Congratz you have passed";
    }

    if (marks < 70 ) {
        cout << "Sorry, you are failed";
    }

    return 0;
}
```

Example: Testing Multiple Cases

What happens if you:

- Assign `int grade` to 60?
- Assign `int grade` to 70?
- Change `grade > 70` in the code above to `grade >= 70`?

Challenge

Compound Conditional Statement

- ▶ Two or more than two conditions are to be tested

```
#include <iostream>
using namespace std;
int main() {
    int grade = 80;
    if (grade >= 70 && grade <= 80){
        cout <<"Grade B ";
    }
    if (grade >= 80 && grade <= 90){
        cout <<"Grade A ";
    }
}
```

Compound Conditional Statement

15

- ▶ Another Example (Your homework)

```
int num = 16;  
  
if (num % 2 == 0 && num > 10) {  
    cout << "Even and greater than 10" << endl;  
}
```


What happens if you:

- Assign num to 8?
- Change `&&` in the code above to `||`?
- Change `==` in the code above to `!=`?

Challenge

```
int my_var = 19;

if (my_var > 15) {
    if (my_var < 20) {
        cout << my_var << endl;
    }
}
```

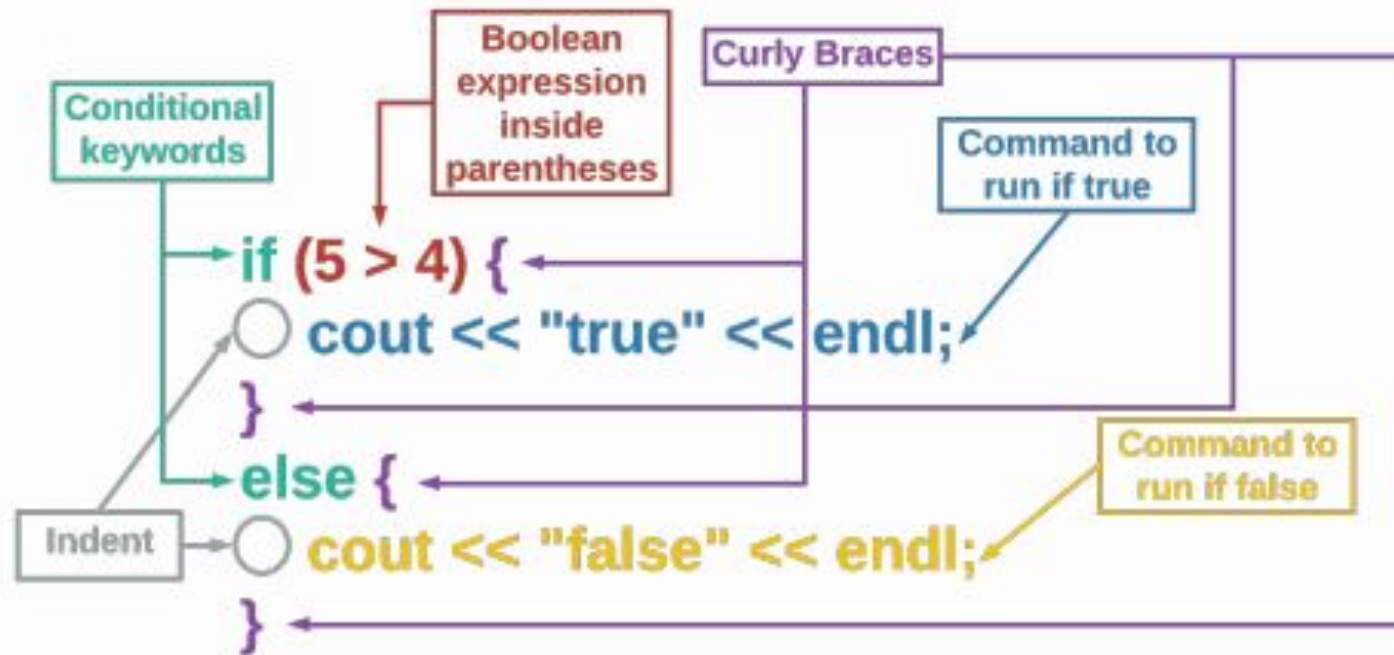
```
int my_var = 19;

if (my_var > 15 && my_var < 20) {
    cout << my_var << endl;
}
```

The code on the left is a **nested** if statement - which means an if statement is *inside* another if statement.

The code with the **compound conditional** (on the right) has fewer lines of code, and is easier for a human to read. In fact, it almost reads like a sentence.

Why use Compound Conditionals



If-Else Syntax

```
#include <iostream>
using namespace std;
int main() {
    if (8 < 7){
        cout << "If statement";
    }
    else{
        cout << "else statement";
    }

    return 0;
}
```

Example

```
if (5 > 4) {  
    cout << "Print me if true" << endl;  
}  
else {  
    cout << "Print me if false" << endl;  
}
```

Indentation is the best practice

Challenge

21

challenge

What happens if you:

- Change 4 in the code above to 6?
- Remove all the curly braces {}?
- Add `cout << "False" << endl;` under `cout << "Print me if false" << endl;` *without* any curly braces {} in the code?
- Add `cout << "True" << endl;` under `cout << "Print me if true" << endl;` *without* any curly braces {} in the code?


```
bool my_bool = true;

if (my_bool) {
    cout << "The value of my_bool is true" << endl; }
else {
    cout << "The value of my_bool is false" << endl; }
```

challenge

What happens if you:

- Assign my_bool to false?
- Assign my_bool to ! true && ! false?

Another if-else example


```
if (is_true) {  
    cout << "1" << endl;  
    cout << "2" << endl;  
}
```

```
if (is_true) {  
    cout << "1" << endl; cout << "2" << endl; }
```

```
if (is_true) { cout << "1" << endl; cout << "2" << endl; }
```

All the codes
work the
same way!

```
#include <iostream>
using namespace std;

int main() {
    int temp = 5;

    if (temp > 35) {
        cout << "It's Hot";
    }
    else if (temp > 20) {
        cout << "It's Warm";
    }
    else if (temp > 10) {
        cout << "It's Cool";
    }
    else {
        cout << "It's Cold";
    }
    return 0;
}
```

Else-if

Challenge

25

- ▶ Write down C++ Code using if statement which reads the marks from the student and then decides the grade of student based on following criteria:
 - ▶ Marks below 60 is F Grade
 - ▶ Marks below 70 is D Grade
 - ▶ Marks below 80 is C Grade
 - ▶ Marks below 90 is B Grade
 - ▶ Marks equal to 100 or less than 100 is A Grade

```
int grade = 62;  
if (grade < 60) {  
    cout << "F" << endl; }  
else if (grade < 70) {  
    cout << "D" << endl; }  
else if (grade < 80) {  
    cout << "C" << endl; }  
else if (grade < 90) {  
    cout << "B" << endl; }  
else if (grade <= 100) {  
    cout << "A" << endl; }
```

If-else-If

Difference

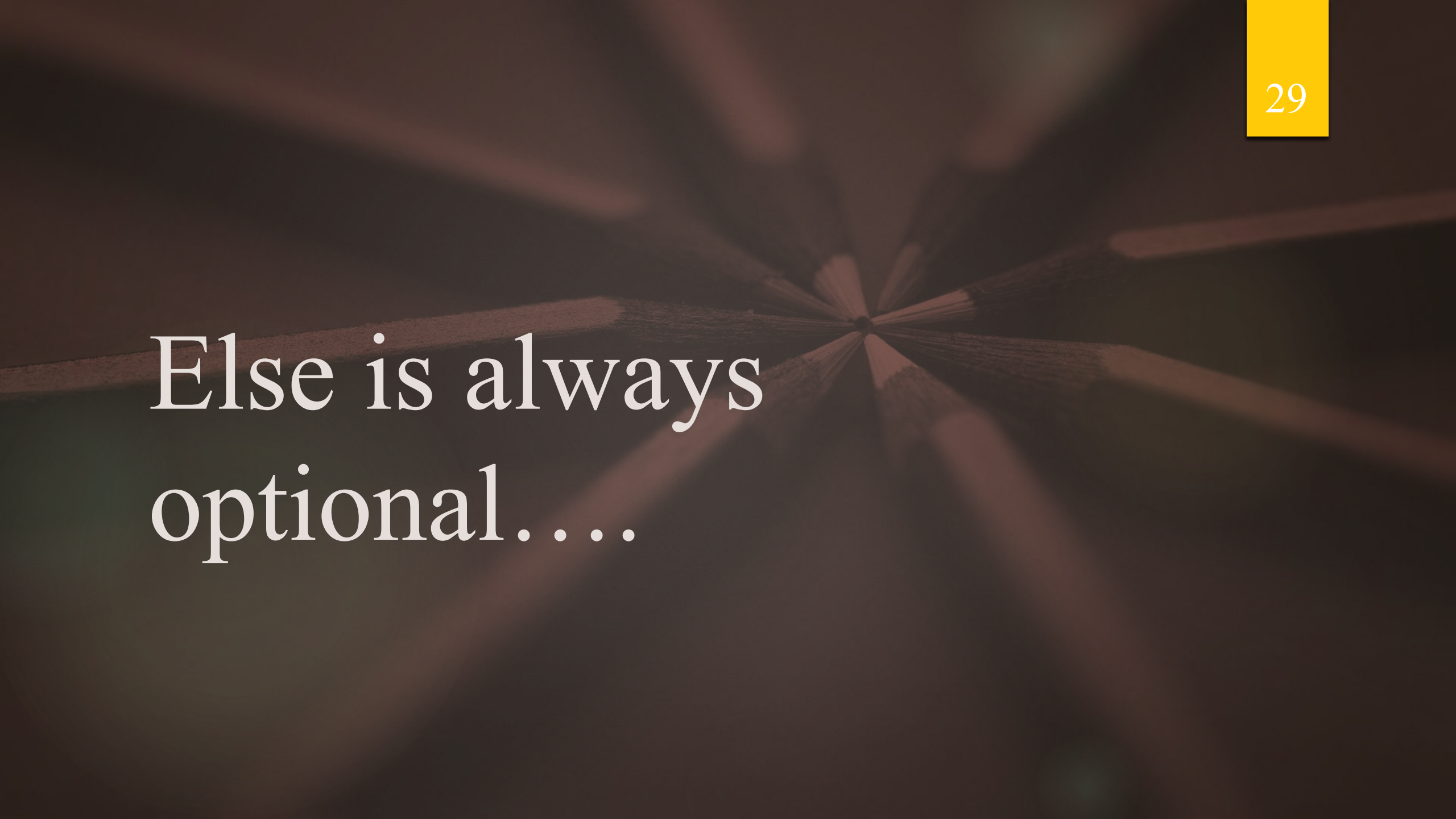
27

Multiple if	if-else if
Each condition is checked independently (compiler evaluates all).	Once one condition is true, the rest are skipped .
Can result in multiple blocks executing if conditions overlap.	Ensures only one block executes .
Slower if you have many conditions, since all are tested.	Faster, avoids unnecessary checks.
Useful when more than one condition can be true at the same time.	Useful when conditions are mutually exclusive (only one should apply).

```
int grade = 62;  
if (grade < 60) {  
    cout << "F" << endl; }  
if (grade >= 60 && grade < 70) {  
    ★ cout << "D" << endl; }  
if (grade >= 70 && grade < 80) {  
    cout << "C" << endl; }  
if (grade >= 80 && grade < 90) {  
    cout << "B" << endl; }  
if (grade >= 90 && grade <= 100) {  
    cout << "A" << endl; }
```

```
int grade = 62;  
if (grade < 60) {  
    cout << "F" << endl; }  
else if (grade < 70) {  
    ★ cout << "D" << endl; }  
else if (grade < 80) {  
    cout << "C" << endl; }  
else if (grade < 90) {  
    cout << "B" << endl; }  
else if (grade <= 100) {  
    cout << "A" << endl; }
```

Difference...



Else is always
optional....

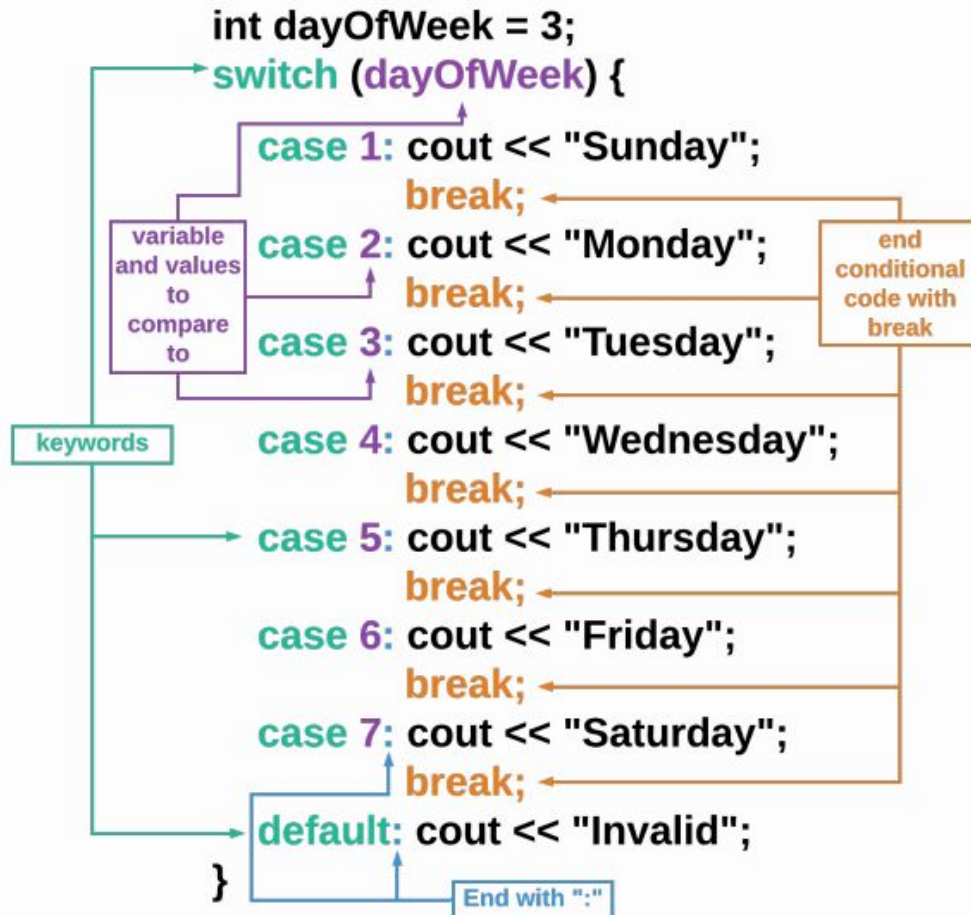
Switch case Statement



The switch case statement is a way to make a decision with multiple possible outcomes.



It evaluates an expression and matches it with a particular value, then it executes set of statements associated with that value



Syntax of Switch-case

- Start with `switch` followed by the variable that is going to be tested in parentheses `()`.
- All of the cases are surrounded by a set of curly braces `{}`.
- Each case is followed by a *numerical* value and a colon `:`.
- After each `:`, write the code that should run if the variable is equal to that case's value.
- After each section of code per case, include `break;`.
- As the very last case, use `default:` to specify what should happen if none of the above cases are true.

Rules for Writing a switch case statement

```
#include <iostream>
using namespace std;

int main() {
    int option = 10;

    switch (option) {
        case 1:
            cout << "You chose Option 1";
            break;
        case 2:
            cout << "You chose Option 2";
            break;
        case 3:
            cout << "You chose Option 3";
            break;
        default:
            cout << "Invalid choice";
    }

    return 0;
}
```

Simple Example

Challenge

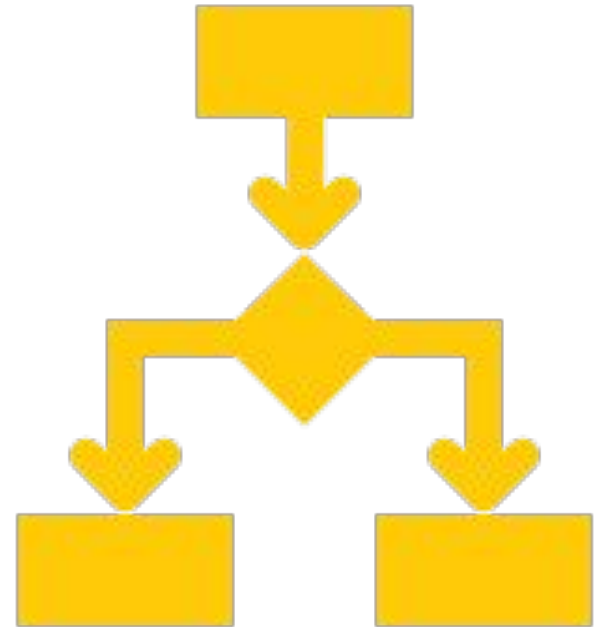
34

- ▶ Input a month in number from 1-12 and display the relevant day is string form.

Switch case vs If-Else-if

- ▶ Switch case and else-if both allows to check for multiple outcomes
- ▶ Then why switch case

35



switch case can only check for equality (e.g. `num == 5`), so if you need to check for a range of values (e.g. `num > 50 && num <= 60`), use else If instead.

```
int grade = 62;
int letterGrade = grade / 10;
switch (letterGrade) {
    case 10: case 9: cout << "A";
                break;
    case 8: cout << "B";
                break;
    case 7: cout << "C";
                break;
    case 6: cout << "D";
                break;
    default: cout << "F";
}
```

```
int grade = 62;
if (grade < 60) {
    cout << "F"; }
else if (grade < 70) {
    cout << "D"; }
else if (grade < 80) {
    cout << "C"; }
else if (grade < 90) {
    cout << "B"; }
else if (grade <= 100) {
    cout << "A"; }
```

#1: Else If is used for ranges of values - Switch Case is for specific values

#2: Else If is used for handling multiple variables

37

Switch case can only compare against values - not variables.

Compare the inputted day of the week with the current day of the week, you would need to use else if.

- Switch case can handle values (`dayOfWeek == "Sunday"`) but not variables (`dayOfWeek == today`).

Rules of the switch case statement in C++

38

The case value must be either int or char type.

There can be any number of cases.

No duplicate case values are allowed.

Each statement of the case can have a break statement. It is optional.

The default Statement is also optional.

Advantages of Switch-case

39

Faster execution speed.

Easier to read than if else if.

Disadvantages of Switch-case

40

Switch case can only evaluate int or char type.

No support for logical expressions.

Have to keep in mind to add a break in every case.

If-else vs Switch Case

Feature	if-else	switch-case
Conditions	Can check complex conditions (>, <, ==, logical operators, ranges).	Checks only equality (==) against constant values.
Data types supported	Works with all data types (int, float, string, bool, etc.).	Works mainly with integers, characters, and enums (not floats).
Readability	Becomes long and harder to read with many conditions.	Cleaner when you have one variable and many possible values.
Execution	Evaluates conditions one by one until match found.	Jumps directly to the matching case (faster in some cases).
Use case	Best for complex conditions or ranges .	Best for menu-driven or option-based programs .

Important points to consider when using switch-case

42

- ▶ No duplicate Case Values
- ▶ Break statement is optional

```
#include <iostream>
using namespace std;

int main()
{
    int var1 = 1;
    int var2 = 0;

    // outer switch
    switch (var1) {
        case 0:
            cout << "Outer Switch Case 0\n";
            break;
        case 1:
            cout << "Outer Switch Case 1\n";
            // inner switch
            switch (var2) {
                case 0:
                    cout << "Inner Switch Case 0\n";
                    break;
            }
            break;
        default:
            cout << "Default Case of Outer Loop";
            break;
    }

    return 0;
}
```

Nested Switch-Case

What is a Loop?

Loop is a statement in C++, which allows us to execute a block of code for n number of times

We need loops to avoid writing repetitive code manually

Each loop has a loop body

The Loop body is the block of code statements written inside the loop

Example Problem to show the need of Loops

45

- ▶ print “Hello World” 10 times

Manual Method

46

```
// C++ program to Demonstrate the need of loops
#include <iostream>
using namespace std;

int main()
{
    cout << "Hello World\n";
    cout << "Hello World\n";
    cout << "Hello World\n";
    cout << "Hello World\n";
    cout << "Hello World\n";
    return 0;
}
```

Using Loops

47

//Loop: Repeats a block of code/number of statements

```
for(int i=0; i<=5; i++){  
    cout<<"Hello";  
}
```

For-Loop

- ▶ **Loop Structure:**
 - ▶ Allows us to write a loop, which executes for a specific number of times
 - ▶ Useful when you know the number of repetitions in advance

Syntax:

```
for (initialization expr; test expr; update expr)
{
    // body of the loop
    // statements we want to execute
}
```

Example

49

```
#include <iostream>
using namespace std;

int main() {
    for (int i = 0; i < 5; i ++ ){
        cout << "Hello World" << endl;
    }
}
```

Example 2

50

```
#include <iostream>
using namespace std;

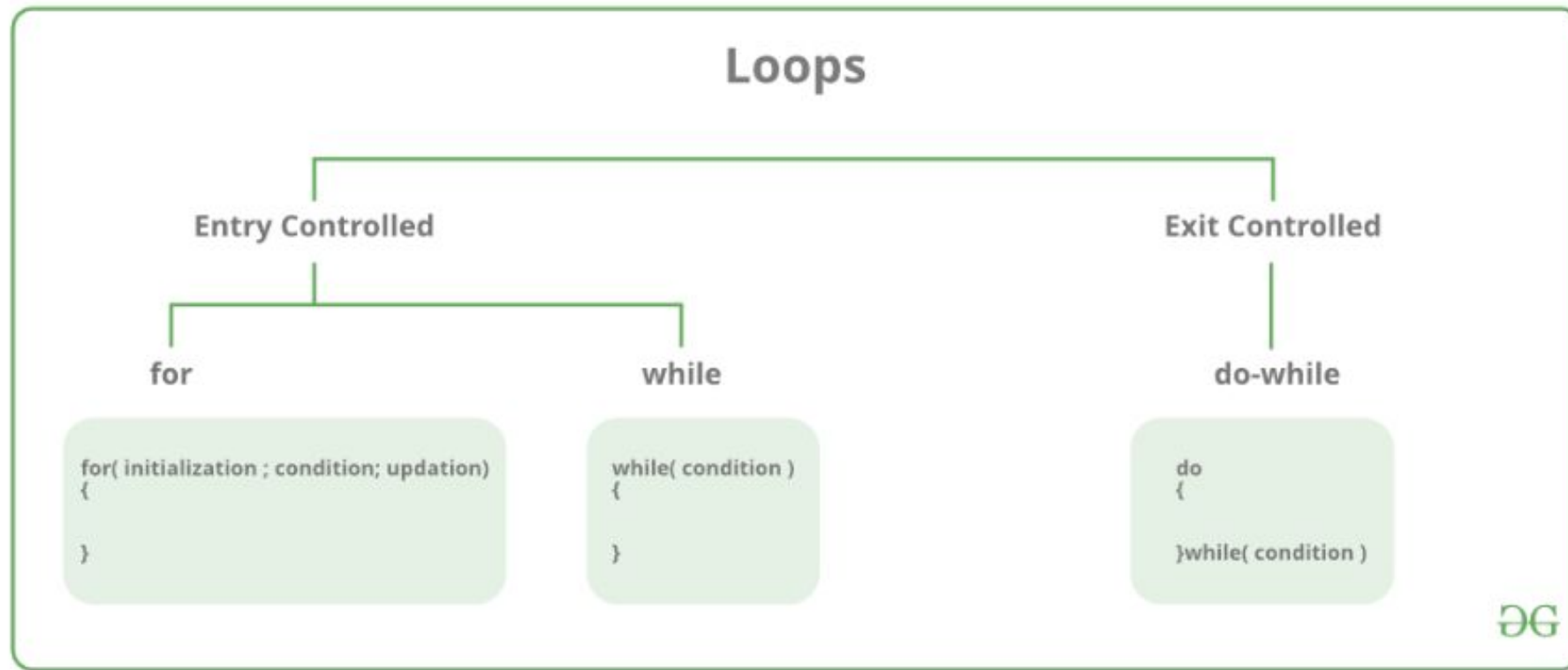
int main() {
    for (int i = 0; i < 5; i ++ ){
        cout << "Hello World" << i << endl;
    }
}
```


Types of Loops

- ▶ Mainly two types of the loops:
 - ▶ Entry Controlled:
 - ▶ Test condition is tested before entering the body of loop
 - ▶ For loop and while loop are examples of entry controlled loops
 - ▶ Exit Controlled :
 - ▶ Test condition is tested after the body of loop, body comes before the test condition
 - ▶ Loop body executes at least once, no matter the condition becomes true/false
 - ▶ Do-While is an example of exit controlled loop

Types of Loops

52



Initialization statement

Executes for once only

Can contain declaration of multiple local variables, limited to the scope of loop

Variables inside main function with same name as the loop variables are hidden during loop execution

Condition

- ▶ This statement gets evaluated ahead of each execution of the loop body.
- ▶ Abort the execution if the given condition get false.



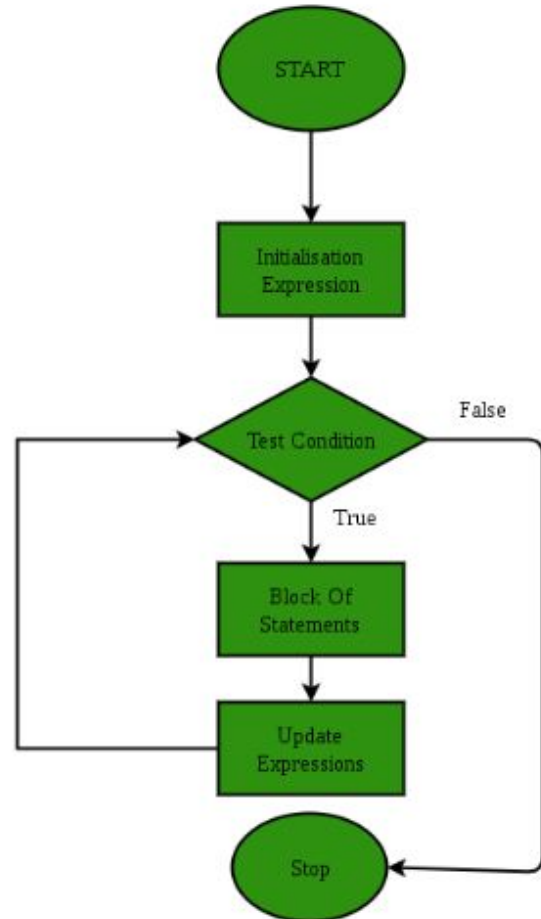


Update expr

THE VALUES OF VARIABLES GET
INCREMENTED/DECREMENTED

Iteration

- ▶ Each single pass through the loop to complete all steps is called iteration of the loop
- ▶ Loops are also called iterative statements



Flowchart of for Loop


```
#include <iostream>
using namespace std;

int main()
{
    for (int i = 0, j = 10, k = 20; (i + j + k) < 100;
        j++, k--, i += k) {
        cout << i << " " << j << " " << k << "\n";
    }
    return 0;
}
```

For loop example:02

```
int main()
{
    int i = 99;
    for (int i = 0; i < 5; i++) {
        cout << i << "\t";
    }
    cout << "\n" << i;
    return 0;
}
```

Example of hiding
declared variables
before a loop is:
C++

for(; ;)

THIS LOOP WILL RUN FOR INFINITE NUMBER OF TIMES.

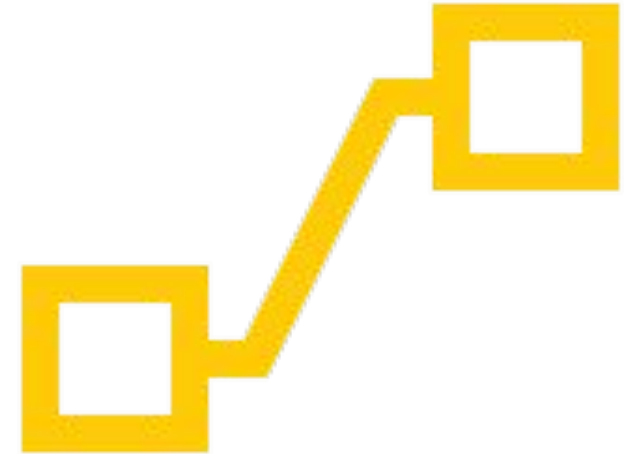


Taking Input Inside a for loop

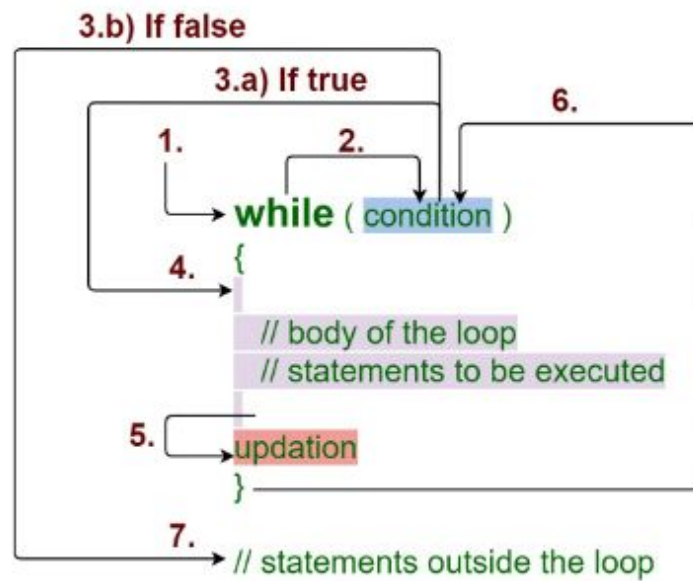
While Loop

- ▶ Used when the number of repetitions/iterations is not known in advance
- ▶ Loop runs and executes a block of code repeatedly until a condition becomes false

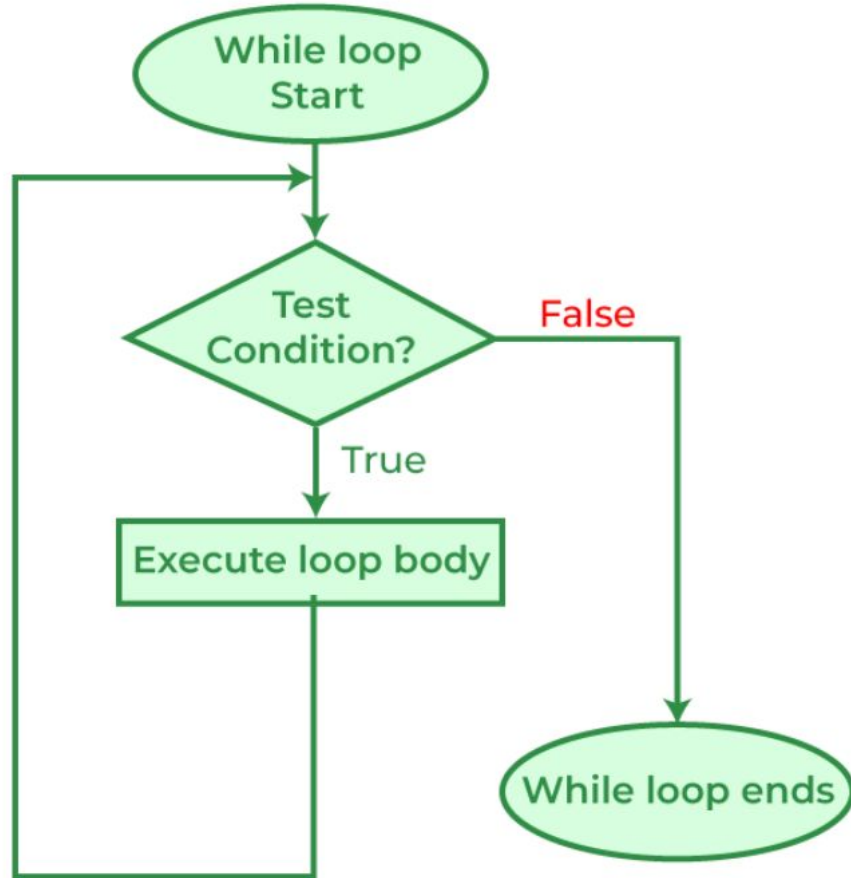
62



While Loop



Syntax of while loop



Flow chart of while Loop


```
// C++ program to illustrate while loop

#include <iostream>
using namespace std;

int main()
{
    // initialization expression
    int i = 1;

    // test expression
    while (i < 6) {
        cout << "Hello World\n";

        // update expression
        i++;
    }

    return 0;
}
```

Example#01 While Loop

```
// C++ program to illustrate while loop
```

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    // initialization expression
```

```
    int i = 1;
```

```
    // test expression
```

```
    while (i > -5) {
```

```
        cout << i << "\n";
```

```
        // update expression
```

```
        i--;
```

```
    }
```

```
    return 0;
```

```
}
```

Example#02

While Loop



Using Multiple Variables and Compound Conditionals in a while loop

Activity

68

- Write code(s) in C++ to print first five even and odd numbers using while loop

Do While Loop

Do while Loop executes a Block of code until a test condition becomes false.



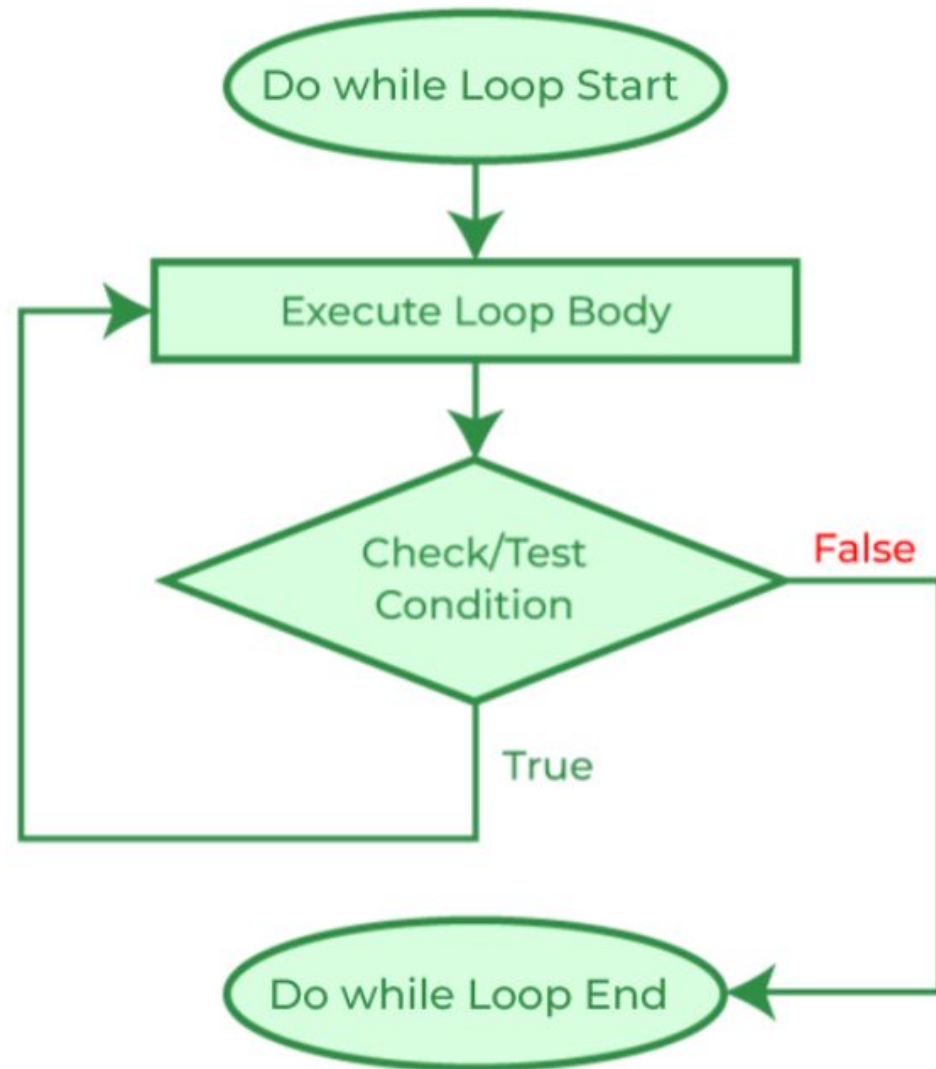
The main difference between a do-while loop and the while loop is in the do-while loop the condition is tested at the end of the loop body.



i.e do-while loop is exit controlled whereas the other two loops are entry-controlled loops.

In a do-while loop, the loop body will *execute at least once* irrespective of the test condition.

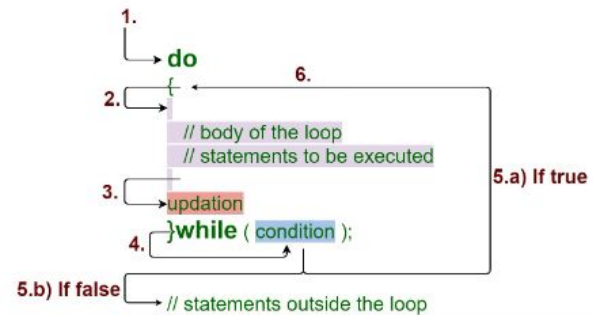
Very
Important!



Do While Loop Flowchart

Syntax

Do - While Loop



```
// C++ program to illustrate do-while loop

#include <iostream>
using namespace std;

int main()
{
    // Initialization expression
    int i = 2;

    do {
        // Loop body
        cout << "Hello World\n";

        // Update expression
        i++;

    }
    // Test expression
    while (i < 1);

    return 0;
}
```

Example#01

do while Loop

```
// C++ program to illustrate do-while loop

#include <iostream>
using namespace std;

int main()
{
    // Initialization expression
    int i = 1;

    do {
        // Loop body
        cout << i << endl;

        // Update expression
        i++;

    }
    // Test expression
    while (i <= 5);

    return 0;
}
```

Example#02 do while loop

```
while(1)
{
    cout<<"This loop will run infinitely";
}

return 0;
}
```

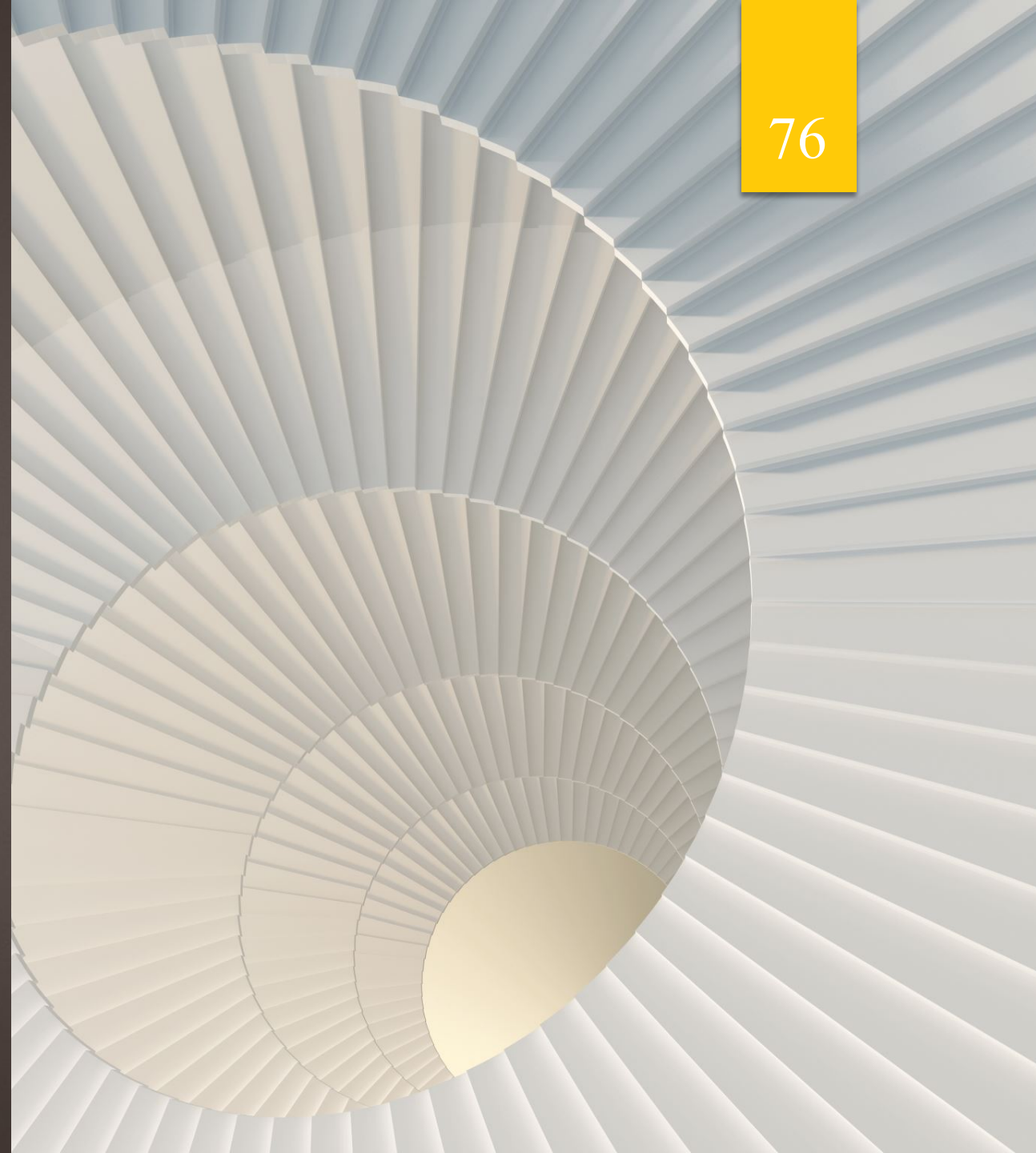
```
do
{
    {
        cout<<"This loop will run infinitely";
    }
}
```

```
while(1);
```

Infinite While and Do while

Nested Loops

- ▶ Nested loop means a [loop statement](#) inside another loop statement. That is why nested loops are also called as “loop inside loop”.




```
for ( initialization; condition; increment ) {  
  
    for ( initialization; condition; increment ) {  
  
        // statement of inside loop  
    }  
  
    // statement of outer loop  
}
```

Syntax of Nested for

```
for(int i=1; i<=5; i++){  
    cout<<"Multiplication table of: "<<i<<endl;  
    for(int j=1; j<=10; j++){  
        cout<<i<<"*"<<j<<"="<<i*j<<"\n";  
    }  
}
```

Example

Assignment

79

- ▶ Implement and Understand Bubble Sort Algorithm using For Loop.

Class Activity

80

Build a simple calculator with a menu that allows the user to perform basic operations (addition, subtraction, multiplication, division). Use a do-while loop to keep the program running until the user chooses to exit.

```
while(condition) {  
  
    while(condition) {  
  
        // statement of inside loop  
    }  
  
    // statement of outer loop  
}
```

Syntax of Nested While

```
int i=1;
int j;

while(i<=5){
    j=1;
    cout<<"Multiplication Table of: "<<i<<endl;
    while(j<=10){
        cout<<i<<"*"<<j<<"="<<i*j<<"\n";
        j+=1;
    }
    i++;
}
```

Nested While


```
do{  
  
    do{  
  
        // statement of inside loop  
    }while(condition);  
  
    // statement of outer loop  
}while(condition);
```

Syntax of Nested do while

Class Activity

84

**

*



No Restriction on
Nesting Level and
type of loop

Jump Statements

- ▶ Jump statements manipulate the flow of program if some conditions are met
- ▶ Four Types:
 - ▶ Break
 - ▶ Continue
 - ▶ Goto
 - ▶ Return

Continue

87

Used inside for, while
or do while loop

Executes some parts of
the loop, while
skipping others

Used with a
conditional statement
which must be present
inside a loop

```
// C++ program to demonstrate the
// continue statement
#include <iostream>
using namespace std;

// Driver code
int main()
{
    for (int i = 1; i < 10; i++) {

        if (i == 5)
            continue;
        cout << i << " ";
    }
    return 0;
}
```

Example

Break



C++ break statement is used with loops and cases of a switch statement.



It forces the loop to terminate, upon meeting a condition



Used with a conditional statement in loop

```
// C++ program to demonstrate the
// break statement
#include <iostream>
using namespace std;

// Driver Code
int main()
{
    for (int i = 1; i < 10; i++) {

        // Breaking Condition
        if (i == 5)
            break;
        cout << i << " ";
    }
    return 0;
}
```

Example

It terminates the entire function, and it is stronger than a break statement

Can be used with or without conditionals

```
// C++ program to demonstrate the
// return statement
#include <iostream>
using namespace std;

// Driver code
int main()
{
    cout << "Begin ";

    for (int i = 0; i < 10; i++) {

        // Termination condition
        if (i == 5)
            return 0;
        cout << i << " ";
    }
    cout << "end";

    return 0;
}
```

Example

Goto statements

- ▶ C++ goto statement can take control to any part of the program
- ▶ C++ goto statement has a label associated with it
- ▶ The label can be written anywhere, before goto or after goto

Syntax of goto

94

Syntax

```
goto label_name;
```

```
.
```

```
.
```

```
.
```

```
label_name:
```

```
// C++ program to demonstrate the
// goto statement
#include <iostream>
using namespace std;

// Driver Code
int main()
{
    int n = 4;

    if (n % 2 == 0)
        goto label1;
    else
        goto label2;

label1:
    cout << "Even" << endl;
    return 0;

label2:
    cout << "Odd" << endl;
}
```

Example